

COS214 Practical 3

Courtesy of

- Azolile Mbanga : u24820360
- Nicole Bare : u25443705
- Patrick Simuyemba : u25632354

Task 1: Event concept and completed architecture

1.1 Event definition

- Our event is called Quest Music Festival. It is a one-day festival that is packed with fun for the whole family. The festival contains several distinct areas, each responsible for providing different services to patrons. At the Main Stage Area, there are a wide range of performers headlining the event on the Main Stage and there is a Security Team available to survey the area and ensure safety in that zone. The Food Court Area contains numerous Food truck units available with a variety of food and refreshments to sort out that grumbling tummy namely a Taco, Mini Donuts and a Chip 'n Dip Food Truck. The Kids Area provides a safe environment for children and our Caretaker team will happily look after them at our Playground. There are ablution facilities available with Men's and Women's Bathrooms equipped with a Cleaner Team to maintain the facilities and each Bathroom contains individual Stall units. There is a Security Team surveying the Main Stage Area and the Gate Entrance Area to keep the event safe throughout the day. There is a gate where patrons will enter and later leave the park to and from. At the Gate Entrance Area there is an Entrance Gate through which ticket-holders will pass and a Security Team to prevent unauthorized entry. A central EventControl hub will coordinate the communication to all the Areas of the festival by issuing notifications when important events occur. Examples include a rain alert, a security breach alert, a festival-opened alert, a festival-closed alert, a Festival capacity reached and a change to the headliner schedule alert. These events may require certain areas or units to react differently to these notifications. This provides meaningful behavior to the Observer pattern as different components can respond polymorphically to the same notification. The Festival is modelled as a Composite structure because the event contains areas with smaller components within them. The Quest Music Festival acts as the root of the structure and contains the Main Stage Area, Food Court Area, Kids Area, Gate Entrance Area and Bathroom Area. These areas are Composite units because they contain other festival components. The Concrete Leaf units are as follows: Main Stage, Security Team, Playground, Caretaker Team, Entrance Gate, Stall, Cleaner Team, Food Truck and do not contain festival components within them. The Composite Areas are as follows: Main Stage Area, Kids Area, Gate Entrance Area, Bathroom Area, Food Court Area. The resulting structure is below.

```
Quest Music Festival
|
|----Main Stage Area:
|      |--Main Stage
|      |--Security Team
|
|----Kids Area:
|      |--Playground
|      |--Caretaker Team
|
|----Gate Entrance Area:
|      |--Entrance Gate
|      |--Security Team
|
|----Bathroom Area:
|      |--Men's Bathroom
|      |--Stall
|      |--Stall
|      |--Stall
```

```

|         |--Women's Bathroom
|         |--Stall
|         |--Stall
|         |--Stall
|         |--Cleaner Team
|
|-----Food Court Area:
|         |--Taco Food Truck
|         |--Chip n Dip Food Truck
|         |--Mini Donuts Food Truck

```

1.2 Identification GoF participants in *Observer*, *Composite*

- Subject: Subject, Observer: Observer, ConcreteSubject: EventControl, ConcreteObserver: EventComponent, FoodTruck, SecurityTeam, MainStage, Playground, CleanerTeam, ToiletStall, EntranceGate, CaretakerTeam, EventGroup
- Component: EventComponent, Composite: EventGroup, Leaf: EventComponent, Client: EventControl
- EventComponent participates in both the Observer and the Composite patterns acting as a bridge between them. As a ConcreteObserver in the Observer pattern, EventComponent implements the update operation used to receive pushed notifications from the Subject and allows each concrete component to react according to its own behaviour. It receives pushed notifications that it would need to act accordingly on. As a Component in the Composite pattern, EventComponent provides the common interface for both EventGroup and EventComponent and allows the client to treat individual units and groups uniformly.
- EventControl participates in both the Observer and the Composite patterns. As a ConcreteSubject in the Observer pattern, it has the implementation of the subject being observed and it implements the functionality to store objects that are observing it and sends update notifications to these objects. As a Client in the Composite pattern, it manipulates the objects that comprise the composite.

1.3 UML diagram completion

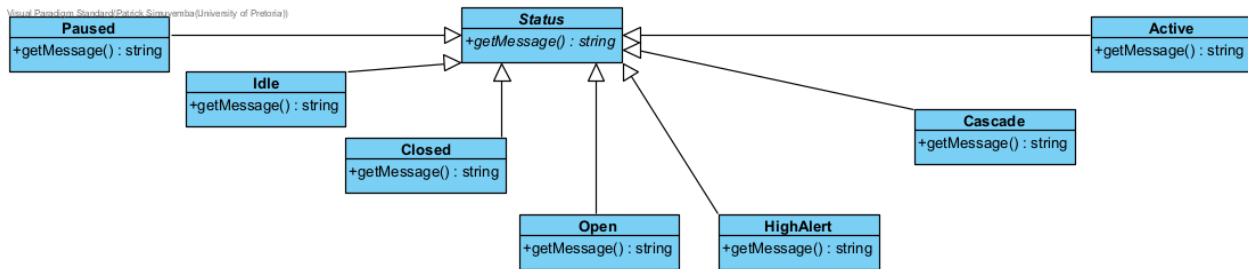


Figure 1: Corrected UML Diagram

2.1 Implementation

2.2 Implementation

2.3 Object Diagram

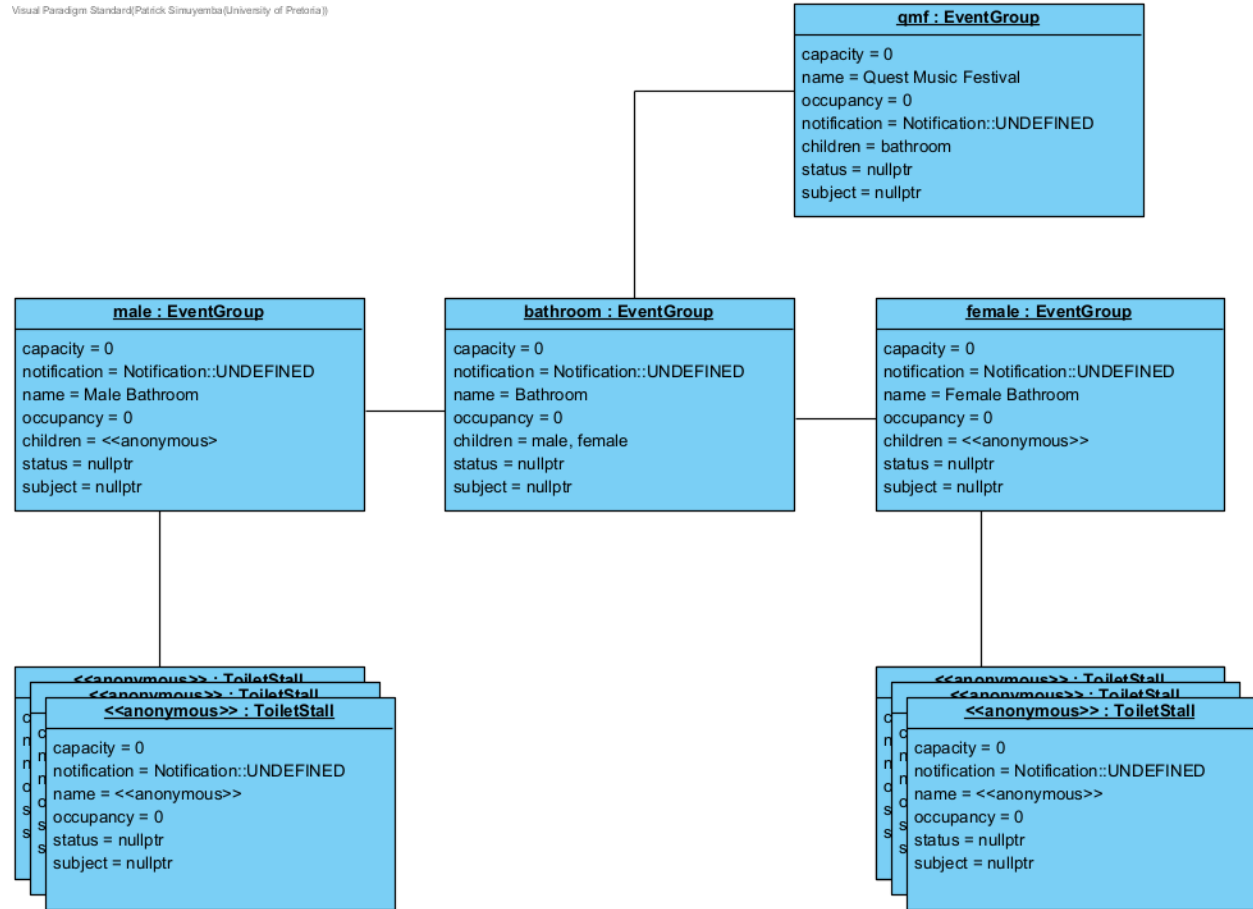


Figure 3: Object diagram of a composite event component

2.4 Memory Management

- **EventGroup** owns its children. The destructor iterates the **children** vector and **deletes** each child. A leaf destructor releases only that leaf. A nested **EventGroup** destructor repeats the same walk, so deleting the root **questFestival** releases the entire owned subtree. Each node has exactly one owning parent, so each object is deleted exactly once.
- Transfer rule: a component belongs to at most one composite at a time. To move a child, the old parent calls **remove(child)**, which unlinks the pointer and does not **delete** it; the new parent then calls **add(child)** and becomes the sole owner. **add** without a prior **remove** would leave two owners and cause a double-free when both composites are destroyed. No **delete** is performed during a transfer.

Task 3: Observer Notification system

3.1 Observer/Subject

- Subject holds a registration list **vector<Observer*> observers** and exposes **attach**, **detach** and **notify**. **EventControl** is the concrete subject.
- Attach policy: a **nullptr** observer is ignored. If the observer is already in the list, the second registration is ignored so that one observer is never notified twice. Otherwise the observer is appended.

- Detach policy: a `nullptr` observer, or an observer that is not in the list, is ignored and is not treated as an error. If the observer is present, every matching entry is erased. This keeps `detach` safe to call more than once during shutdown.

3.2 Observer

- `Observer` declares `virtual void update() = 0`. Concrete observing behaviour lives on `EventComponent` and its subclasses: `update` pulls the latest notice from `EventControl` and then calls `determineStatus`, so each leaf reacts according to its own mapping of the Notification update.
- Lifetime policy: observers detach themselves. When an `EventComponent` is destroyed it calls `subject->detach(this)` if it still has a subject pointer. The subject does not own observers and must not `delete` them; it only forgets the pointer. That pairing means a destroyed observer cannot remain in the registration list as a dangling pointer. Subjects therefore never notify an object whose lifetime has already ended.

3.3 Notice/Order types

- Ordinary operational change: `OPEN`, `CLOSE`, `SCHEDULE_CHANGE`.
- Capacity-related change: `CAPACITY_ALERT`.
- Safety-related change: `WEATHER_ALERT`, `SECURITY_ALERT`.
- Each leaf maps those notices onto a different status (`Cascade`, `Open`, `Closed`, `Paused`, `Active`, `Idle`, `HighAlert`).

3.4 Composite level class

- `EventGroup` is the Composite-level class that both receives a notice and forwards it downward. When `update` runs on a group, the group pulls the current notice, sets its own status to `Cascade`, and then run `update()` on the observers registered on that group.
- `EventControl` notifies `questFestival`; `questFestival` cascades to `bathroomArea`; `bathroomArea` cascades to `mensBathroom` and then to `stallM1`. The same path can be read as `EventControl` → `Quest Music Festival` → `Bathroom Area` → `Stall`. Each level receives the notice from above and notifies interested observers below it. The `Cascade` status records that a group is forwarding rather than applying a leaf-specific open/close state of its own.

3.5 pull vs push model

- Pull mode: `Subject::notify()` calls `Observer::update()` with no arguments. The observer then queries the subject: `EventComponent::update` calls `subject->getNotification()` and `subject->getCapacity()`, stores those values, and then computes a new `Status` through `determineStatus`.
- Trade-off: pull keeps the `update` signature stable when extra subject state is added, because each observer requests only the fields it needs. The cost is that every observer must hold a subject pointer and know which getters to call. A push model would pass the notice (and capacity) into `update(...)` directly, which is simpler for observers but forces the subject interface to change whenever the payload grows.
- Exact state transferred by pull: the current `Notification` (`OPEN`, `CLOSE`, `WEATHER_ALERT`, `SCHEDULE_CHANGE`, `CAPACITY_ALERT` or `SECURITY_ALERT`) and the integer `capacity`. `EventControl` uses occupancy internally to decide when to emit `CAPACITY_ALERT`; `EntranceGate` also reads `capacity/occupancy` for display after the pull.

Task4: Event rules and dynamic behaviour

4.4

- The three original features are event-specific and don't create a good object because each feature belongs to the relevant leaf.

Task 5: Sequence diagram portfolio

5.1 SD1 - Building and registering part of the event

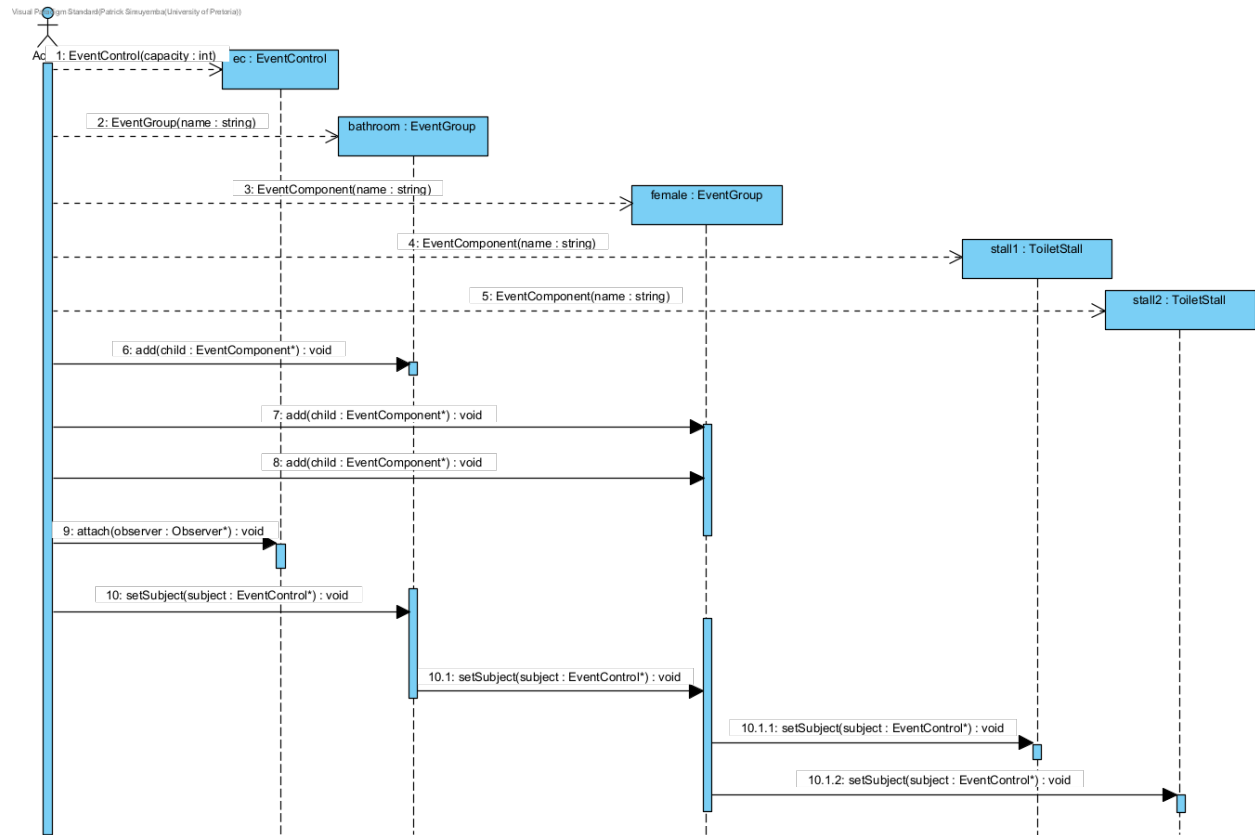


Figure 4: Image showing the sequence of events for building and registering part of an event

5.2 SD2 - Cascading event notification

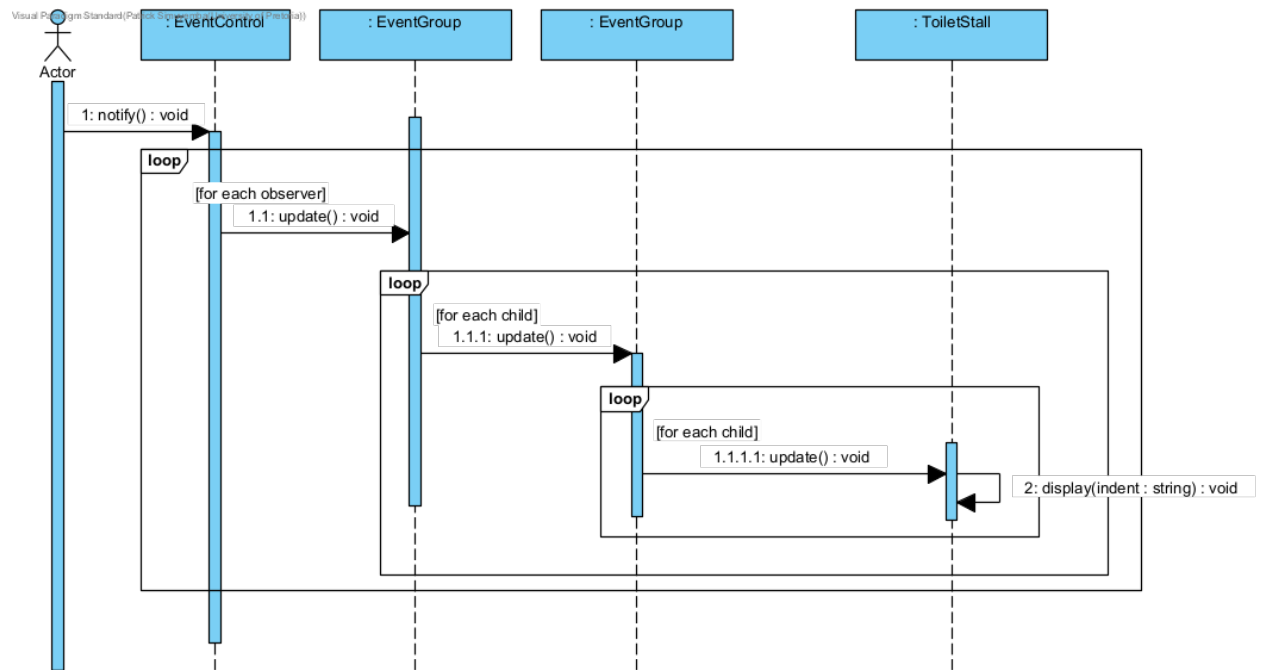


Figure 5: Sequence diagram for cascading notification

5.3 SD3 - Conditional event response and Composite behaviour

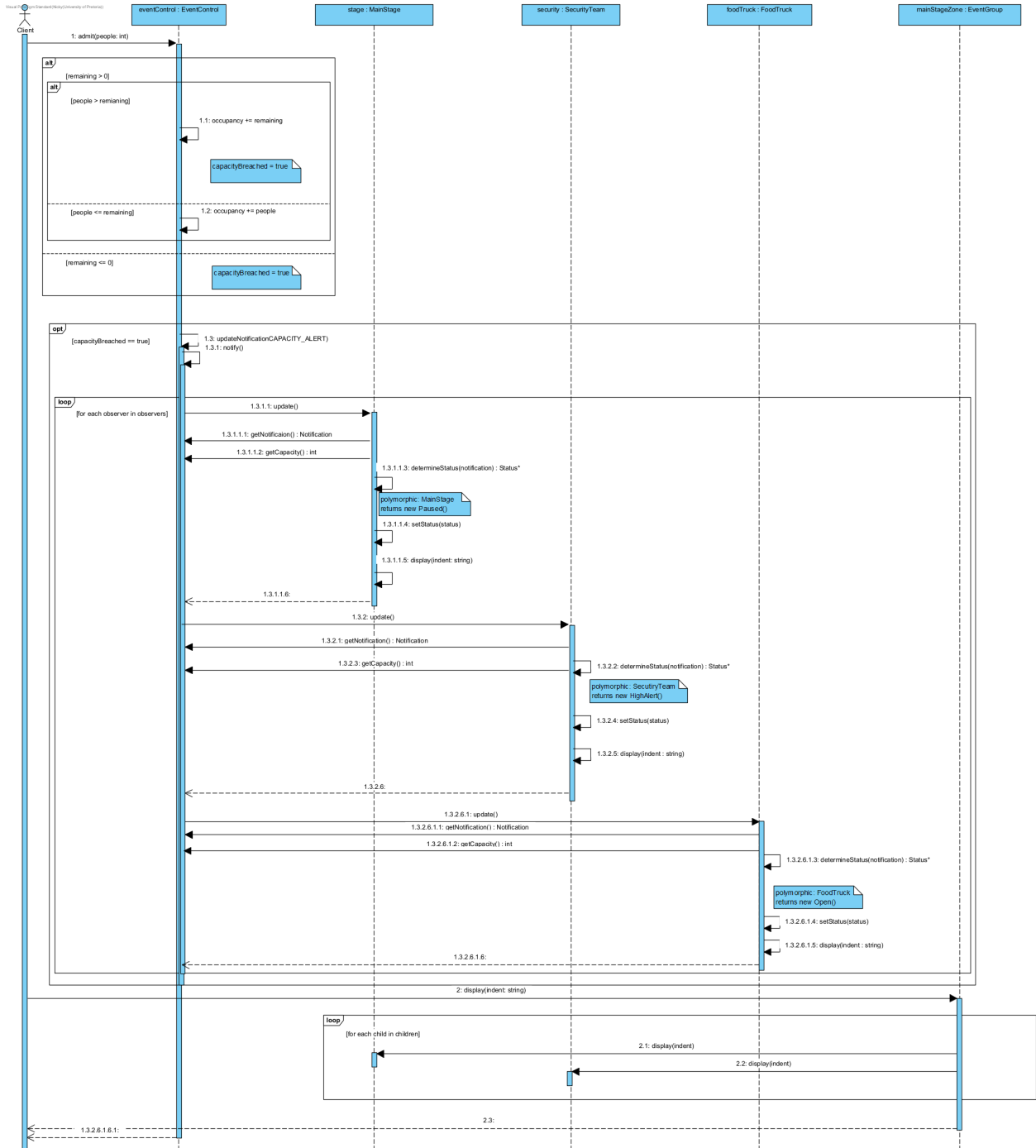


Figure 6: Sequence diagram of Conditional event response and Composite behaviour

5.4 SD4 - Your signature event scenario

Changes were integrated through `git merge`, and several real conflicts arose and were resolved rather than avoided. `EventControl::admit()` was edited independently on the Task 4 branch and on `dev` at the same time, producing a conflict that Patrick resolved by hand to preserve both the capacity-threshold logic and the Task 4 notification behaviour (commit `96fa59d`). A second conflict, on `EventComponent` itself, was resolved by Azolile during the final integration pass once every task's work had been merged into `dev` (commit `710a542`). Three commits stand out as evidence of genuine collaboration rather than a single end-of-project dump: `96fa59d` and `710a542`, the two conflict resolutions described above, in which two members' independent changes to the same class were reconciled by hand; and `c06dc25`, in which Azolile fixed the Observer notify infinite loop discovered only after Task 4's dynamic behaviour was exercising the merged Composite/Observer code at runtime, showing the team debugging shared code together after integration rather than treating each member's merge as final.

We did not use formal GitHub Pull Requests for this practical; branches (`dev` and `feature/task4-dynamic-behaviour`) were merged directly once reviewed between members over calls/emails, and coordination was tracked through commit messages and branch names rather than a PR review thread. The repository history spans 23–28 August 2026 with commits from all three members throughout, rather than a single late submission.

Task 8: Intergration and demonstration